Solving the Lorenz ODE System Using Optimal ANN Architectures

By

Ikramul Hasan Moral (0112230489)

Md. Abu Bakar (0112230200)

Samiur Rahman Omlan (0112230195)

Fariha Islam (0112230431)

Md. Touhidul Islam (0112230435)

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

June 8, 2026

Department of Computer Science and Engineering
United International University

Abstract

Nonlinear coupled ordinary differential equations (ODEs) occur often in physical modeling. In most cases, their analytical solutions cannot be obtained. The traditional approach to solving such problems is numerical methods such as the Runge-Kutta family, which are not free of computational cost and mesh dependence. The Lorenz-1960 ODE system is a simplified meteorological model that serves as the test case in this study. This study investigates whether a standard artificial neural network (ANN), without physics-informed loss functions or operator learning configurations, can approximate the solution to this coupled system. Accurate reference data are generated using the Runge-Kutta method and the SciPy solver. Several feedforward ANN structures are then trained on this reference data. Then, change the network structure by adjusting the number of hidden layers, the number of neurons per layer, and the activation functions. Lastly, test the models by checking the mean squared error, training convergence, and the time required to compute them.

Acknowledgements

First of all thanks to ALMIGHTY ALLAH for giving us the strength and ability to complete this project successfully.

We sincerely express our gratitude to our honorable mentor, Dr. Muhammad Nomani Kabir, Professor, Department of CSE, United International University, for his guidance, support, and encouragement throughout this project.

We also thank our respected course teacher, Dr. Md. Motaharul Islam, Professor, Department of CSE, United International University, for his kindness and valuable teachings.

We are also grateful to all the faculty members who supported and guided us in completing this project.

Last but not the least, we thank our parents and family members for their constant support, love, and encouragement.

Table of Contents

Table of Contents iv

List of Figures v

List of Tables vi

1 Introduction 1
- 1.1 Project Overview . . . 1
- 1.2 Motivation . . . 2
- 1.3 Objectives . . . 3
- 1.4 Methodology . . . 3
- 1.5 Project Outcome . . . 4
- 1.6 Organization of the Report . . . 5

2 Background 6
- 2.1 Preliminaries . . . 6
  - 2.1.1 Ordinary Differential Equations . . . 6
  - 2.1.2 The Lorenz-1960 System . . . 6
  - 2.1.3 Classical Numerical Methods . . . 7
  - 2.1.4 Artificial Neural Networks . . . 7
  - 2.1.5 ANN-Based ODE Solving . . . 7
  - 2.1.6 Physics-Informed Neural Networks . . . 8
  - 2.1.7 Network Architectures . . . 8
- 2.2 Literature Review . . . 9
  - 2.2.1 Similar Applications . . . 9
  - 2.2.2 Related Research . . . 9
- 2.3 Gap Analysis . . . 12
- 2.4 Summary . . . 13

3 Project Design 14
- 3.1 Requirement Analysis . . . 14
  - 3.1.1 Functional and Nonfunctional Requirements . . . 14
  - 3.1.2 Context Diagram . . . 15

3.1.3 Data Flow Diagram Level 1 16
3.1.4 UI Design 16
3.2 Detailed Methodology and Design 17
3.2.1 Reference Solution Generation 17
3.2.2 Dataset Preparation 17
3.2.3 Model Family and Architecture (Phase 1) 18
3.2.4 Optimizer Comparison (Phase 2) 18
3.2.5 Loss Function and Training Protocol 18
3.2.6 Evaluation Metrics 19
3.2.7 Alternative Solutions Considered 19
3.3 Project Plan 20
3.4 Task Allocation 21
3.5 Summary 22

4 Implementation and Results **23**

5 Standards and Design Constraints **24**
5.1 Compliance with the Standards 24
5.2 Design Constraints 24
5.3 Cost Analysis 24
5.4 Complex Engineering Problem 24
5.4.1 Complex Problem Solving 24
5.4.2 Engineering Activities 26
5.4.3 Knowledge and Attitude Profile 27
5.5 Summary 28

6 Conclusion **29**

References **32**

List of Figures

1.1 Ordinary differential equations describe rate-of-change processes throughout science, engineering, and modern machine learning. The Lorenz-1960 system studied in this report belongs to the weather and climate family. . . 2
1.2 Ordinary differential equations underpin modern scientific machine learning. A numerical solver produces reference data that trains an artificial neural network; the trained network then acts as a fast surrogate that predicts the state at any time without re-solving the equations. This report follows the plain data-driven ANN route, in contrast to the physics-informed and operator-learning alternatives below. 2
1.3 Methodology Diagram 4

2.1 Feedforward ANN used in this study. A single scalar input, time t, is propagated through a stack of hidden layers to predict the Lorenz-1960 state $(x(t), y(t), z(t))$. The number of hidden layers, the neurons per layer, and the activation function are the axes compared in this work [1]. 8
2.2 Physics-Informed Neural Network for the Lorenz-1960 system. The network outputs $(\tilde{x}, \tilde{y}, \tilde{z})$ are differentiated to form the ODE residuals r_x, r_y, r_z, whose mean square becomes a physics loss added to the data loss [2]. The plain-ANN approach adopted in this report deliberately omits the physics term ($\lambda = 0$), training on the data loss alone. 9

3.1 Context diagram of the research pipeline. The Research Team supplies configurations and receives outputs. The Academic/Research Community receives the published findings and methodology. 16
3.2 Data Flow Diagram (Level 1) showing the five internal processes of the research pipeline. 16
3.3 36-week FYDP Gantt chart using task IDs. 20

List of Tables

2.1 Comprehensive Literature Gap Analysis 12

3.1 Functional Requirements of the Research Pipeline 15
3.2 Nonfunctional Requirements of the Research Pipeline 15
3.3 Phase 1 Architecture Search Grid 18
3.4 Evaluation Metrics Used for Each Trained Model 19
3.5 Task and Milestone IDs Used in the Gantt Chart 21
3.6 Task Allocation Across Team Members and FYDP Segments 21

5.1 Mapping with complex problem solving. 24
5.2 Mapping with complex engineering activities. 26
5.3 Mapping with knowledge and attitude profile. 27

Chapter 1

Introduction

This chapter introduces the research topic, provides an overview and motivation for the study, and outlines the main objectives and expected outcomes. It also outlines the methodology and briefly describes how the rest of the report is organized.

1.1 Project Overview

Ordinary Differential Equations (ODEs) describe how a system change with over time. They are widely used in fields such as physics, engineering, and mathematics. In many real-world problems, particularly those involving complex or nonlinear equations, it is often not possible to obtain exact solutions. That is why methods such as Euler's method and Runge-Kutta are used to obtain approximate solutions. In this project, we examine a particular system consisting of three nonlinear equations known as the Lorenz-1960 system (i.e., one of the first meteorological models). The system is small, but complex enough to make a good testing ground for various solution methods. The Lorenz-1960 system is a great candidate for evaluating how well ANNs learn and predict behavior. Classical numerical methods address these problems step by step. Also they are typically accurate, but they are likely very slow and require more computational cost. We use numerical methods like Runge-Kutta to generate data that can be used to train artificial neural networks to learn how variables change over time. The ANN learns to match its predictions with the results of the numerical solver during training. After training, the ANN can quickly predict results for any time without solving the equations again. Therefore, the primary aim of this project is to solve the Lorenz-1960 system and to see how well a plain ANN can learn its behavior and which ANN architecture provides optimal results.

Figure 1.1 shows the breadth of fields where such rate-of-change models appear.

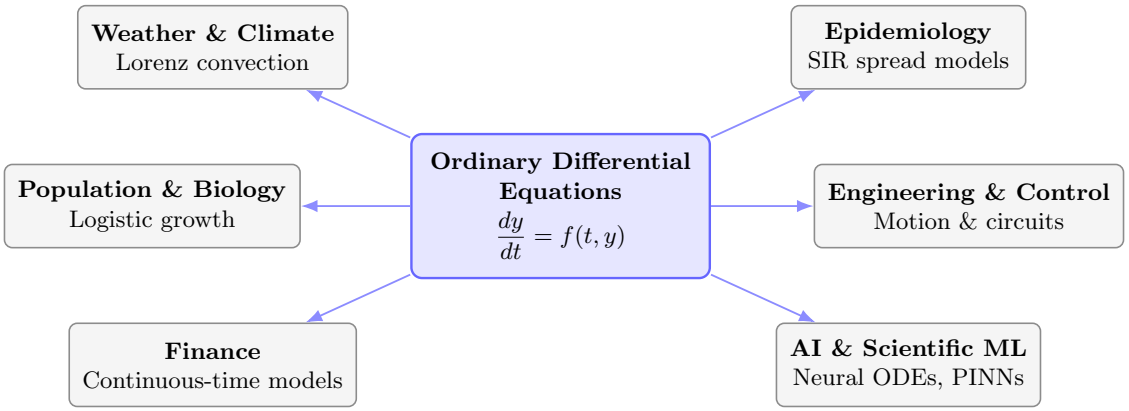

Figure 1.1: Ordinary differential equations describe rate-of-change processes throughout science, engineering, and modern machine learning. The Lorenz-1960 system studied in this report belongs to the weather and climate family.

1.2 Motivation

This project is motivated by both research and practical sides. Classic numerical methods are reliable and accurate. They work well for directly simulating equations. However, they can be slow when we need to run them many times. A trained artificial neural network (ANN), on the other hand, can solve the equation on its own. It makes predictions and acts like a copy of the real system. Many studies now use neural networks to solve differential equations, including physics-informed neural networks (PINNs), operator learning, and ANN-based approximators. This makes the topic both relevant and interesting to explore. Another reason for this project is the learning and hands-on benefits it offers. It helps build skills in solving ordinary differential equations (ODEs). It also teaches how to build and use neural network models. The project involves running experiments and studying the results. Comparing different methods is helpful. Even if simple ANNs are not as powerful as advanced methods, they help us understand their strengths and weaknesses.

Figure 1.2 summarizes how numerical solvers and neural networks combine in this data-driven workflow.

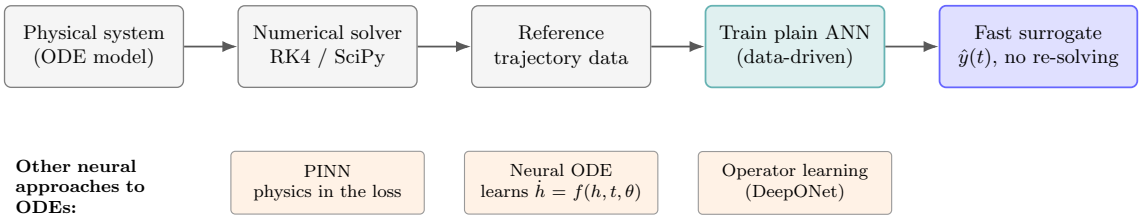

Figure 1.2: Ordinary differential equations underpin modern scientific machine learning. A numerical solver produces reference data that trains an artificial neural network; the trained network then acts as a fast surrogate that predicts the state at any time without re-solving the equations. This report follows the plain data-driven ANN route, in contrast to the physics-informed and operator-learning alternatives below.

1.3 Objectives

The primary objectives of this project are:

- To solve the Lorenz equation, which is a coupled and nonlinear system.
- To use ODE concepts, numerical methods like Runge-Kutta and SciPy solvers, and an ANN for solving the Lorenz equations.
- To create accurate data using the Runge-Kutta method and use it to validate the ANN performance and compare its results.
- To compare different ANN architectures by changing the number of layers, the number of neurons per layer, and trying different activation functions to find the most accurate model.
- To evaluate the model's performance using mean squared error, learning stability, and computational cost, and see if a plain ANN can learn the Lorenz equation.
- To compare the results with PINN based methods from other research to understand how this approach performs compared to recent techniques.

1.4 Methodology

We are following the "Waterfall Methodology."Here, each phase is completed sequentially in a fixed order. It is suitable because our goals and requirements are clear from the start. Here, every step produces outputs that guide the next. This makes the process structured, simple to track, and easy to manage.

First, during the Literature Review phase, we studied the Lorenz-1960 equation and its numerical solution methods, like RK4 and SciPy solvers. We also reviewed multiple ANN designs, activation functions, and training methods.

Next, in the Concept Development phase, the research problem and objectives were clearly defined. We focused on approximating the Lorenz-1960 equation using ANNs. We also worked on setting up the network layers, choosing the number of neurons, activation functions, learning rates, and training epochs, and generating training and validation data.

In the Design phase, we developed the ANN models and training workflow, covering data preprocessing and optimized architecture. We established a numerical baseline using RK4 and SciPy to compare results. Assessment measures, visualization tools, and statistical tests were also defined to guarantee fair model comparison.

In the Implementation phase, the numerical solvers and ANN models were implemented in Python . All models were trained on generated datasets . Accuracy, training time, number of parameters, and convergence were recorded. Each model was executed systematically according to the design.

Then, in the Testing phase, we measured each ANN model's performance and predicted its performance using mean square error. We tested generalization on new data and analyzed the models' sensitivity to different parameter settings. The best models were compared with results from published studies.

Finally, in the Deployment phase, we put all results, plots, and analyses into the final project report. We organized and documented the code to make it easy to reproduce our work. We included instructions for repeating the experiments and retraining the models. We also gave our final recommendations for the best ANN architecture. This phase ensures that the project is complete and ready.

For better understanding, a diagram of our methodology is given below:-

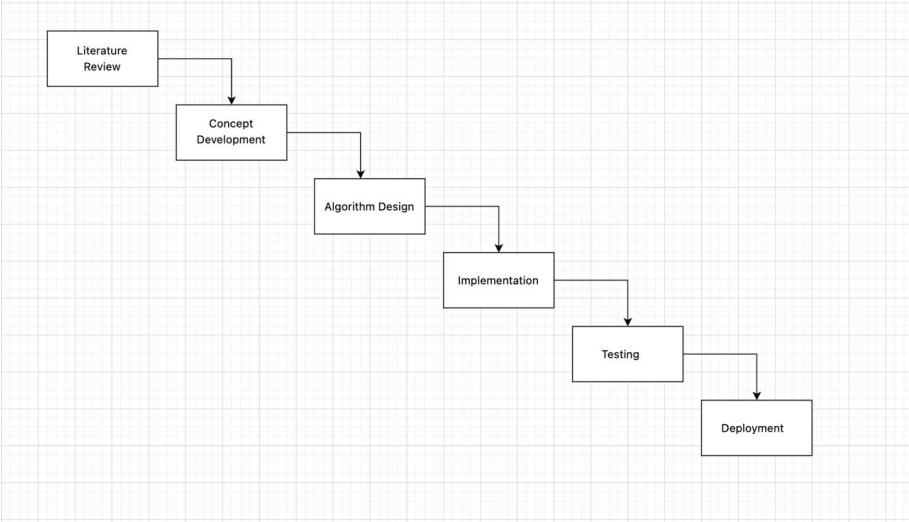

Figure 1.3: Methodology Diagram

1.5 Project Outcome

The expected outcome of the project will be the clear and the justified determination of the most appropriate architecture for the ANN in estimating the solution to the Lorenz-1960 system of Ordinary Differential Equations (ODEs) within the scope of the experiment. This project must also include a numerical reference solution to the ODEs, the establishment of ANN baselines, and a comparative analysis of the results.

The project will also contribute to the current understanding of the aptitude and limitations of ANN models in the approximation and solution of differential equations. If the ANN models are able to get a satisfactory level of accuracy in approximating the solution to the ODEs, the project will have demonstrated the capability of a simple ANN architecture to effectively act as an alternative numerical solver. This would suggest that a plain ANN, without the need for physics-informed loss terms or traditional mesh-dependent numerical methods, can on its own approximate ODE solutions with sufficient accuracy

while being computationally less expensive in terms of inference time and the avoidance of iterative re-solving for each query. But if the models cannot achieve satisfactory accuracy in approximating the solution to the ODEs then the study will clearly outline the constraints of simple ANN architectures and underscore the need for advanced methods like PINNs.

The expected outcome of the project will therefore be more than just the coding and experiment itself but will also involve a research based understanding of how ANN architectural choices like depth, width, and activation function can impact solution accuracy for coupled nonlinear ODE systems.

1.6 Organization of the Report

The rest of this report is outlined as follows. Chapter 2 outlines the theoretical background and literature review on Ordinary Differential Equations (ODE) solving, artificial neural networks, and existing neural methods for differential equations. Chapter 3 discusses about the project design, requirements, methodological plan, and task assignment. Chapter 4 outlines the implementation & experiment design used in this research. Chapter 5 outlines the results and analysis on the tested ANN structures. Finally, Chapter 6 concludes this report by focusing the main results, limitations, and potential directions for the future research.

Chapter 2

Background

This chapter covers the foundational knowledge you need to understand our work. We start with the mathematical and computational preliminaries. Then we review the existing literature on ANN-based ODE solvers and similar applications. We identify the research gaps that motivate our study. Finally, we wrap up with a brief summary.

2.1 Preliminaries

This section provides the necessary background knowledge to understand the rest of the report.

2.1.1 Ordinary Differential Equations

An ODE describes how a quantity changes with respect to a single independent variable. A general first-order ODE takes the form:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0 \tag{2.1}$$

where $y(t)$ is the unknown function, t is the independent variable, and y_0 is the initial condition.

2.1.2 The Lorenz-1960 System

The Lorenz-1960 system [3] consists of three coupled nonlinear ODEs for meteorological modeling. We target the formulation from [4] (Section 4.2):

$$\begin{aligned}
\frac{dx}{dt} &= kl\left(\frac{1}{k^2+l^2} - \frac{1}{k^2}\right) yz \\
\frac{dy}{dt} &= kl\left(\frac{1}{l^2} - \frac{1}{k^2+l^2}\right) xz \\
\frac{dz}{dt} &= \frac{kl^2}{2}\left(\frac{1}{k^2} - \frac{1}{l^2}\right) xy
\end{aligned} \tag{2.2}$$

with $k = 2$, $l = 1$, initial conditions $x(0) = 0.5$, $y(0) = 0.75$, $z(0) = 1$, and $t \in [0, 1]$. Its nonlinear, coupled dynamics make it an excellent benchmark for numerical solvers.

2.1.3 Classical Numerical Methods

Euler's method approximates the solution via a first-order Taylor expansion:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n) \tag{2.3}$$

where h is the step size. It is simple but accumulates errors quickly.

The fourth-order Runge-Kutta method (RK4) evaluates the derivative at multiple points per step for higher accuracy:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{2.4}$$

Both methods are mesh-dependent, require re-solving for every new initial condition, and struggle with stiff equations.

2.1.4 Artificial Neural Networks

A feedforward ANN (Multilayer Perceptron) consists of an input layer, hidden layers, and an output layer. Each neuron computes:

$$a_i(\mathbf{x}) = \sigma\left(\mathbf{w}^{(i)} \cdot \mathbf{x} + b^{(i)}\right) \tag{2.5}$$

where $\mathbf{w}^{(i)}$ are weights, $b^{(i)}$ is bias, and σ is an activation function [5]. The Universal Approximation Theorem [5, 6, 7] guarantees that a network with at least one hidden layer can approximate any continuous function to arbitrary accuracy. Common activation functions include:

- **Sigmoid**: $\sigma(x) = \frac{1}{1+e^{-x}}$, maps inputs to $(0, 1)$
- **Tanh**: $\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, maps inputs to $(-1, 1)$
- **ReLU**: $\sigma(x) = \max(0, x)$, efficient but non-smooth at zero
- **GELU/Swish**: smooth ReLU approximations with better gradient flow

Training minimizes a loss function $L(\theta)$ over network parameters θ using gradient descent. Backpropagation computes gradients via the chain rule, and optimizers like Adam adapt learning rates per parameter [8, 9].

2.1.5 ANN-Based ODE Solving

Instead of stepping through the domain, ANNs learn a continuous solution function over the entire domain. The standard approach [10, 11, 12] constructs a trial solution:

$$\psi_t(x, \mathbf{p}) = A(x) + F(x, N(x, \mathbf{p})) \tag{2.6}$$

where $A(x)$ satisfies the initial/boundary conditions exactly, and $N(x, \mathbf{p})$ is the neural network output. The loss function measures how well this trial solution satisfies the ODE:

$$L(\mathbf{p}) = \frac{1}{N} \sum_{i=1}^{N} \left[\frac{d\psi_t(x_i, \mathbf{p})}{dx} - f(x_i, \psi_t(x_i, \mathbf{p})) \right]^2 \tag{2.7}$$

Minimizing this loss yields a differentiable, closed-form approximate solution without requiring knowledge of the exact answer.

2.1.6 Physics-Informed Neural Networks

PINNs embed governing equations directly into the loss function [4, 6, 9, 13]:

$$L(\theta) = L_{\text{data}} + \lambda_1 L_{\text{physics}} + \lambda_2 L_{\text{IC/BC}} \tag{2.8}$$

The physics loss uses automatic differentiation to compute derivatives of the network output. This enables training with sparse or no data, but balancing the loss terms requires careful hyperparameter tuning.

2.1.7 Network Architectures

To better understand the differences in methodology, we present the standard architectures used in recent studies. Figure 2.1 illustrates a typical Artificial Neural Network (ANN) architecture used for predicting system parameters without physics-informed constraints [1]. This standard feedforward structure relies entirely on data-driven learning through backpropagation.

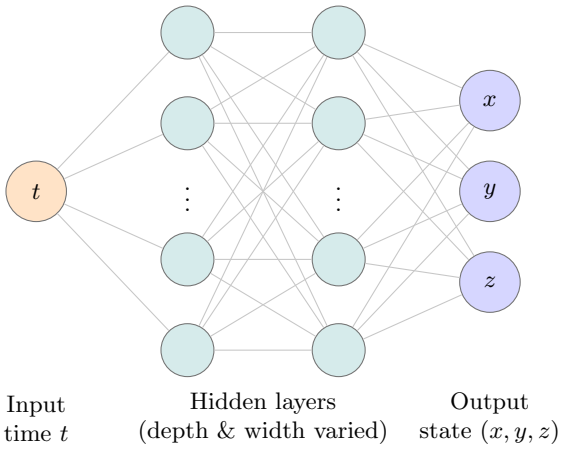

Figure 2.1: Feedforward ANN used in this study. A single scalar input, time t, is propagated through a stack of hidden layers to predict the Lorenz-1960 state $(x(t), y(t), z(t))$. The number of hidden layers, the neurons per layer, and the activation function are the axes compared in this work [1].

In contrast, Figure 2.2 demonstrates a Physics-Informed Neural Network (PINN) architecture. In this approach, the standard ANN output is further constrained by physical laws—specifically, the governing differential equations of the system—which are embedded directly into the loss function [2].

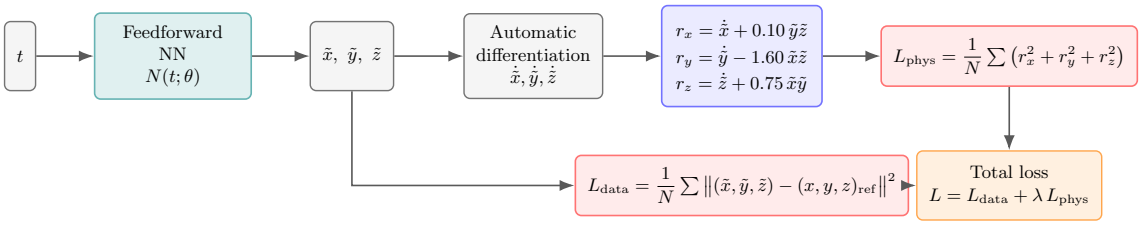

Figure 2.2: Physics-Informed Neural Network for the Lorenz-1960 system. The network outputs $(\tilde{x}, \tilde{y}, \tilde{z})$ are differentiated to form the ODE residuals r_x, r_y, r_z, whose mean square becomes a physics loss added to the data loss [2]. The plain-ANN approach adopted in this report deliberately omits the physics term ($\lambda = 0$), training on the data loss alone.

2.2 Literature Review

2.2.1 Similar Applications

Dubey et al. [8] used an ANN (tanh activation) to solve coupled first-order ODEs with an error of 2.88×10^{-3} in a maximum of 120 iterations. A comparable PyTorch-based ANN was used by Audu et al. [14], which converged in 200–1500 iterations. Okereke et al. [12] bypassed backpropagation by calculating the weights based on regression with Gaussian Radial Basis Functions. The ODEN framework developed by Lau and Werth [5] demonstrates that NNs outperform RK4 on rapidly oscillating solutions, with errors of $\sim 10^{-4}$. On a damped vibration equation, Mall and Chakraverty [11] compared arbitrary versus regression-based weight initialization and they found that regression-based weights reduce maximum deviation from 3.67% to 1.47%. Murugesh et al. [15] used neural networks with block numerical methods to directly solve higher-order ODEs and obtaining errors near machine precision ($\sim 10^{-10}$). Pratama et al. [16] surveyed three ANN-based PDE solvers (PyDEns, NeuroDiffEq, Nangs) and used 3 hidden layers with 32 neurons and tanh activation. NeuroDiffEq uses the Trial Approximate Solution methodology, which suggests that initial conditions are hardcoded in the ansatz. This is a pure ANN method that is applied to our research.

2.2.2 Related Research

Physics-Informed Approaches

PinnDE was proposed by Matthews and Bihlo [4], which solved the Lorenz-1960 system using a DeepONet (4 hidden layers, 40 nodes, tanh, Adam optimizer). Their DeepONet is trained to learn the operator $G(u_0)(t, x)$ which provides solutions given initial conditions

over $[0, 1]$. This is the closest baseline to our target problem, but it relies on physics-informed loss terms. Babni et al. [6] generalized the trial construction $\tilde{u}(x, \theta) = P(x) + Q(x)N(x, \theta)$ by replacing $Q(x)$ with a flexible $\varphi(x)$, proving the error bound $|N_\varphi(t, \theta) - u(t)| \leq e^{\kappa|t-t_0|}L_\varphi(\theta)$. This verifies that the smaller the loss, the smaller the solution error. Audu et al. [9] compared PINN and ANN models using six first-order ODE tests. Their PINN had an error of 10^{-8} to 10^{-10} which was much better as compared to ANN. This raises the issue of whether architecture optimization can bridge this gap or not.

Pure ANN Approaches

The NeuroDiffEq Trial Approximate Solution is physics-free. Pratama et al. [16] found that NeuroDiffEq can train without physics loss terms which is consistent with our methodology. But they benchmarked only on the heat equation which is linear. Dubey et al. [8] claim that plain ANNs can solve coupled ODEs but used a fixed architecture. Okereke et al. [12] removed backpropagation through weight computation instead of polynomial fitting. Audu et al. [14] achieved an error of 10^{-4} with a sigmoid ANN on three first-order ODEs. All these studies do not examine the implications of building patterns on performance.

Architecture and Loss Function Studies

Mall and Chakraverty [11] made comparisons between 4, 5, and 6 hidden nodes with random and regression-based weights. Weights that are based on regression are always better compared to random weights. After 6 nodes they found that accuracy remains the same. They applied only sigmoid activation on simple ODEs. Lau and Werth [5] discovered that sigmoid / tanh need less training in comparison to ReLU / ELU. Xavier initialization speeds up the convergence and Adam is better than SGD. They did not vary systematically depth and width. Xiong [17] arranged seven constructions (polynomial, exponential, hyperbolic, logarithmic, logistic, sigmoid, softplus) of loss functions. Exponential is best when applied to the law of Newton cooling and logarithmic is best when it is applied to the motor suspension system.

Lorenz System Studies

Aslam et al. [18] have used ANN + PSO-NNA hybrid optimization to solve Lorenz equations (Neurons=10, sigmoid, N=90, independent runs=100). They are directed to a diverse Lorenz parameterization (σ, ρ, β), are based on meta heuristic optimization and do not compare architectures.

Neural ODEs

Chen et al. [19] proposed neural ODEs where the derivative function $\frac{dh(t)}{dt} = f(h(t), t, \theta)$ that is parameterized with a neural network and trained using the adjoint method efficient

for backpropagation. This sample is quite different than ours where we learn the dynamics (f) while neural ODEs find the approximation of the solution ($y(t)$) itself.

Hybrid Approaches

Murugesh et al. [15] proposed a Neural-ODE Hybrid Block Method with the update rule:

$$y(t_{k+1}) = y(t_k) + h \sum_{j=0}^{m-1} \omega_j f_\theta(t_{k-j}, y_{k-j}) + \sum_{i=0}^{m-1} \alpha_i F(t_{k-i}, y_{k-i}) \tag{2.9}$$

where ω_j and α_j balance neural versus numerical contributions. It achieved machine-precision errors on harmonic oscillators and the Van der Pol equation.

2.3 Gap Analysis

Table 2.1 summarizes what each reviewed paper covers and where gaps exist.

Table 2.1: Comprehensive Literature Gap Analysis

Paper	**Methodological Approach**	**Target Differential Equations**	**Architectural Investigation**	**Lorenz-1960 Application**	**Plain ANN Implementation**
PinnDE [4]	PINN + DeepONet: Integrates physics laws and operator networks into the loss function.	Solves both ODEs and PDEs using governing physics.	No systematic study of network structural impact.	Evaluated specifically as a complex dynamical system.	Uses physics-informed logic, not a plain ANN.
ANN Survey [16]	PyDEns / ANN: Standard networks for field estimation.	Primary focus on solving Partial Differential Equations (PDEs).	General overview without specific layer/neuron testing.	The study does not address the Lorenz model.	Covers standard ANN architectures for PDEs.
Neural ODEs [19]	Neural ODE: Derivative represented as a neural network.	Models continuous-time dynamics for ODEs.	Focuses on functional flexibility, not structural depth.	Not specifically tested on the Lorenz system.	Relies on specialized ODE-solver integration.
Aslam et al. [18]	ANN + PSO-NNA: ANN with Particle Swarm Optimization.	Applied to solving Ordinary Differential Equations.	Optimization focuses on weights, not architectural depth.	Directly applied to Lorenz-based chaotic dynamics.	Uses heuristic optimization instead of plain training.
ODEN [5]	Plain ANN: Standard feedforward network approach.	General application for approximating ODE solutions.	Provides a limited (partial) study on hidden layers.	Tested on other benchmarks, not Lorenz-1960.	Utilizes a standard, plain ANN configuration.
Mall & Chakraverty [11]	ANN: Implementation of neural networks for equations.	Targets solutions for ODEs in engineering.	Includes a detailed study on network architecture.	Application restricted to simpler, non-chaotic ODEs.	Employs a basic feedforward ANN structure.
Xiong [17]	ANN: Simple neural solver for dynamic systems.	Focused on solving Ordinary Differential Equations.	Does not explore the impact of different layer counts.	Does not address the chaotic Lorenz-1960 model.	Uses a traditional, data-driven ANN model.
Babni et al. [6]	PINN: Physics-Informed Neural Networks approach.	Solves ODEs by embedding equations in the loss function.	No investigation into depth or width performance.	Not applied to the meteorological Lorenz model.	Strictly uses physics-informed constraints.
Dubey et al. [8]	Plain ANN: Feedforward network trained on numerical data.	Applied to various Ordinary Differential Equations.	Lacks a systematic study of network architecture.	No specific testing on the Lorenz system.	Uses a standard numerical data training pipeline.
Okereke et al. [12]	ANN: Basic neural network models for ODEs.	Aimed at solving standard ODE systems.	Does not perform structural performance analysis.	No mention or testing of the Lorenz-1960 system.	Follows a traditional, plain ANN approach.
Audu et al. (2024) [14]	Plain ANN: Usage for initial value problems in ODEs.	Specifically handles ODE solution approximation.	No detailed evaluation of layers or neurons.	Focused on other common types of ODEs.	Relies on a standard feedforward structure.
Audu et al. (2026) [9]	PINN vs ANN: A comparative study of two methods.	Analysis restricted to Ordinary Differential Equations.	Lacks an in-depth architecture comparison study.	Not applied to the Lorenz-1960 model.	Discusses ANN only in comparison to PINN.
Murugesh et al. [15]	Hybrid: Combines different neural modeling techniques.	Solves ODEs using integrated hybrid approaches.	Architecture optimization is not the primary focus.	Does not utilize the Lorenz-1960 test case.	Incorporates external optimization methods.
Lagaris et al. [10]	ANN: Trial-solution method using a feedforward network.	Solves ODEs, ODE systems, and PDEs.	No systematic study of layers or neurons.	Does not test the Lorenz-1960 system.	Plain feedforward ANN trained on the ODE residual.
Raissi et al. [13]	PINN: Embeds governing equations in the loss function.	Targets forward and inverse nonlinear PDEs.	No systematic study of depth or width.	Not applied to the Lorenz-1960 model.	Uses physics-informed constraints, not a plain ANN.
Lorenz [3]	Analytical: Reduces the vorticity equation to three ODEs.	Derives the coupled nonlinear Lorenz-1960 ODEs.	Predates neural network architecture studies.	Defines the original Lorenz-1960 system.	Uses no neural network of any kind.
Hornik et al. [7]	Theory: Proves the universal approximation property.	Not concerned with differential equations.	Shows one hidden layer suffices for approximation.	Does not address the Lorenz-1960 system.	Covers feedforward networks in general, not a solver.
Our Work	**Plain ANN: Traditional feedforward network.**	**Direct solution of coupled ODE systems.**	**Comprehensive study of hidden layers and density.**	**Primary test case for predicting chaotic dynamics.**	**Purely data-driven ANN without physics aids.**

The literature review reveals five specific gaps that our research addresses:

1. **No architecture study for the Lorenz-1960 system:** Matthews and Bihlo [4] solve the Lorenz-1960 system but use a fixed DeepONet architecture without optimization. Aslam et al. [18] study a different Lorenz parameterization. No existing paper systematically compares ANN architectures on the Lorenz-1960 system.

2. **No PINN vs. plain ANN comparison on Lorenz-1960:** Audu et al. [9] compared PINN and ANN on simple first-order ODEs. No study makes this comparison on a nonlinear, coupled ODE system like Lorenz-1960.

3. **No systematic activation function study for ODE solving:** Pratama et al. [16] use tanh throughout without ablation. Lau and Werth [5] note that sigmoid and tanh outperform ReLU but do not test modern alternatives like GELU or Swish. Mall and Chakraverty [11] use only sigmoid. No study compares activation functions (tanh, ReLU, sigmoid, GELU, Swish) on a coupled ODE system.

4. **No systematic depth and width study for ODE solution quality:** Mall and Chakraverty [11] test 4, 5, and 6 nodes on simple ODEs. Lau and Werth [5] observe that architecture should match equation complexity. But no paper systematically varies both depth (2–6 layers) and width (20–100 neurons) to find the optimal configuration for a Lorenz-type system.

5. **Limited generalization studies:** Most papers train for a single initial condition. The ability to generalize across different initial conditions remains largely unexplored for coupled nonlinear ODE systems.

Our research fills these gaps by performing a systematic architecture comparison on the Lorenz-1960 ODE system using plain ANNs. We vary depth, width, and activation functions while benchmarking against RK4 reference solutions and the PINN-based results from Matthews and Bihlo [4].

2.4 Summary

This chapter established the necessary background for our research. We reviewed the mathematical foundations of ODEs, the Lorenz-1960 system, classical numerical methods, and ANN-based solving approaches. The literature review covered 17 research papers spanning physics-informed methods, pure ANN approaches, architecture studies, Lorenz system solvers, Neural ODEs, and hybrid techniques.

The gap analysis reveals that no existing study performs a systematic architecture comparison of plain ANNs on the Lorenz-1960 ODE system. Existing works either use fixed architectures, focus on simpler equations, or rely on physics-informed loss terms. Our research directly addresses this gap by varying network depth, width, and activation functions to identify the optimal ANN architecture for solving the Lorenz-1960 system as an initial value problem.

The next chapter presents our project design, including the detailed methodology, experimental setup, and evaluation plan.

Chapter 3

Project Design

This chapter explain how we designed the experiment. This chapter will represent the requirements of our pipeline needs to meet, the context it operates in and the two-phase methodology we use for architecture search and optimizer comparison.Also the project plan with task ownership across the team.

3.1 Requirement Analysis

Our pipeline has clear requirements. It must produce specific outputs, hit defined quality targets, and operate within fixed constraints. This is a computational research project, not a software product. So the requirements focus on reproducibility, numerical accuracy, and disciplined experimentation.

3.1.1 Functional and Nonfunctional Requirements

The functional requirements tell what the pipeline does at each stage, from generating the reference solution to producing the final comparison report. The nonfunctional requirements set the quality bar. Reproducibility and reference accuracy are the two strictest constraints.

Table 3.1: Functional Requirements of the Research Pipeline

ID	Requirement
FR1	The system shall generate a high-accuracy numerical reference solution for the Lorenz-1960 ODE system using the fourth-order Runge-Kutta method and SciPy DOP853, evaluated over $t \in [0, 1]$ at 1001 uniformly spaced points.
FR2	The system shall partition the reference dataset into training and validation subsets using an 80/20 random split with a fixed random seed.
FR3	The system shall construct feedforward ANN models with systematically varied depth (1 to 4 hidden layers), width (20, 50, or 100 neurons per layer), and activation function (tanh, ReLU, sigmoid, GELU, Swish).
FR4	The system shall train all Phase 1 models under identical conditions using the Adam optimizer and record convergence behaviour for each run.
FR5	The system shall conduct a Phase 2 optimizer comparison on the three best Phase 1 architectures, training each with Adam, L-BFGS, and SGD with momentum.
FR6	The system shall evaluate each trained model using MSE, MAE, maximum absolute error, training time, and inference time against the RK4 reference.
FR7	The system shall produce comparison tables, solution trajectory plots, and error curves for every evaluated model.

Table 3.2: Nonfunctional Requirements of the Research Pipeline

ID	Requirement
NFR1	All experiments shall be fully reproducible using a single fixed random seed applied to weight initialization and data splitting.
NFR2	The numerical reference solution shall maintain solver agreement of RMSE below 10^{-10} between the custom RK4 implementation and SciPy DOP853.
NFR3	All results, model configurations, and metrics shall be logged systematically so that an independent reader can verify the reported findings.

3.1.2 Context Diagram

Figure 3.1 shows the research pipeline as one bounded system with two external entities. The first is the research team. We configure the experiments, trigger each run, and collect the resulting models and metrics. The second is the academic and research community. They consume the published findings, reuse the methodology, and build on the comparative results. Every internal step, from reference generation to final reporting, sits inside the pipeline. Nothing else lives outside the system boundary.

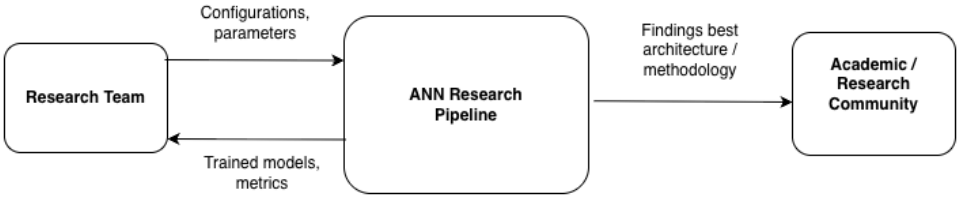

Figure 3.1: Context diagram of the research pipeline. The Research Team supplies configurations and receives outputs. The Academic/Research Community receives the published findings and methodology.

3.1.3 Data Flow Diagram Level 1

Figure 3.2 breaks the research pipeline into five sequential processes. Process P1 generates the reference trajectory using the RK4 and SciPy DOP853 solvers. P2 preprocesses that trajectory by performing the 80/20 split and applying z-score normalization to the output targets. P3 trains the ANN models, first across the full Phase 1 grid, then across the Phase 2 optimizer comparison. P4 evaluates every trained model against the reference solution using the metrics from FR6. P5 compares the results, picks the best architecture, and produces the final report.

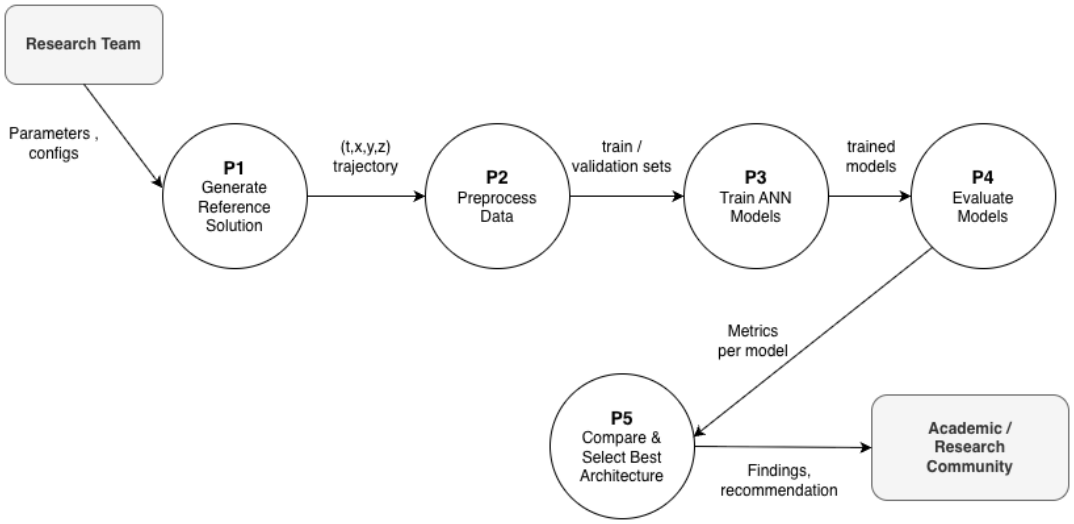

Figure 3.2: Data Flow Diagram (Level 1) showing the five internal processes of the research pipeline.

3.1.4 UI Design

This project has no user interface. We run everything from Python scripts and Jupyter notebooks. Results are stored as figures, tables, and serialized model files. UI design does not apply here.

3.2 Detailed Methodology and Design

We organized the methodology as a two-phase comparative experiment. Here is the logic.

- **Phase 1 – Architecture Search (60 runs):** We vary depth (1–4 hidden layers), width (20, 50, 100 neurons), and activation function (tanh, ReLU, sigmoid, GELU, Swish) while holding the optimizer fixed at Adam. This tells us which architectural choices matter most.
- **Phase 2 – Optimizer Comparison (9 runs):** We take the three best architectures from Phase 1 and retrain each one with Adam, L-BFGS, and SGD with momentum. Everything else stays the same. This tells us how much the optimizer contributes on top of the architecture.

This separation prevents the two comparisons from confounding each other. Any improvement in Phase 2 comes from the optimizer, not from a change in network structure.

3.2.1 Reference Solution Generation

We compute the reference solution from the Lorenz-1960 system with $k = 2$, $l = 1$, initial conditions $x(0) = 0.5$, $y(0) = 0.75$, $z(0) = 1$, and time interval $t \in [0, 1]$, as defined in Chapter 2. We use two independent solvers to make sure the numbers are right. The first is a custom fourth-order Runge-Kutta integrator with step size $h = 10^{-3}$, evaluated on a uniform grid of 1001 points. The second is the SciPy `solve_ivp` routine using DOP853 (eighth-order Dormand-Prince) with rtol $= 10^{-10}$ and atol $= 10^{-12}$, sampled on the same grid. We require agreement between both solvers at RMSE below 10^{-10}. Then we use the RK4 trajectory as the ground truth for all ANN training and evaluation. This dual-solver check catches accidental errors in either implementation and gives us a defensible reference for the rest of the study.

3.2.2 Dataset Preparation

The reference trajectory gives us 1001 rows of the form $(t, x(t), y(t), z(t))$. That is our training set. We split it into 80% training (about 800 points) and 20% validation (about 201 points) using a random split with seed 42. We do the split once before training any model. Every ANN configuration sees the same training points, the same validation points, and the same presentation order.

After splitting, we standardize each of the three target variables using its training-set mean and standard deviation. This z-score normalization puts x, y, and z on a comparable scale so that the loss function does not get dominated by whichever variable has the widest range. But here is the important part: we report all errors after unnormalization. The published metrics stay in the original physical units of the system.

3.2.3 Model Family and Architecture (Phase 1)

Each candidate ANN is a plain feedforward network. It takes scalar time t as input and predicts the three-dimensional state vector $(x(t), y(t), z(t))$. The hidden structure varies along three axes: number of hidden layers, number of neurons per hidden layer, and activation function. Table 3.3 lists the values we explore in Phase 1.

The Cartesian product of these axes gives us $4 \times 3 \times 5 = 60$ distinct architectures. We train each one independently under identical optimization settings. The output layer is always linear with three units. We add no skip connections, no normalization layers, and no regularization. This bare-bones design is intentional. It keeps the comparison focused on the three architectural axes from our research question.

Table 3.3: Phase 1 Architecture Search Grid

Axis	**Values**
Depth (hidden layers)	1, 2, 3, 4
Width (neurons per layer)	20, 50, 100
Activation function	tanh, ReLU, sigmoid, GELU, Swish
Total configurations	$4 \times 3 \times 5 = 60$

3.2.4 Optimizer Comparison (Phase 2)

Phase 2 takes the three architectures with the lowest combined validation MSE from Phase 1 and retrains each one with two additional optimizers: L-BFGS and SGD with momentum. Everything else stays the same as Phase 1. We reuse the Adam result from Phase 1 as the third comparison point. That gives us $3 \times 3 = 9$ Phase 2 runs.

Why these optimizers? L-BFGS is a strong baseline for smooth, low-dimensional regression problems. Researchers commonly cite it as effective for ANN-based ODE solvers. SGD with momentum serves as a classical reference. This phase isolates the optimizer's effect from the network architecture's effect. This separation would not be achievable if optimizer selection were included in the Phase 1 search grid.

3.2.5 Loss Function and Training Protocol

The training loss is the mean squared error between the normalized predicted state and the normalized reference state, computed jointly over all three output components. We add no initial-condition penalty and no physics-based residual. This is consistent with the plain-ANN scope from Chapter 2 and avoids the loss-balancing problem that plagues physics-informed approaches.

Here are the training settings.

- Phase 1 uses Adam with learning rate 10^{-3} and default momentum parameters.
- Each model trains for at most 10,000 epochs.

- Early stopping monitors validation loss with a patience of 500 epochs and restores the best-validation weights when training ends.
- We use full-batch gradient descent. The training set is small enough that full-batch is appropriate, and it removes minibatch noise as a confounding variable across configurations.
- We set the PyTorch random seed to 42 before constructing each model, so weight initialization is reproducible too.

3.2.6 Evaluation Metrics

We evaluate each trained model against the RK4 reference using five metrics. Table 3.4 lists them. MSE is the primary indicator because it matches the training objective and allows direct comparison across all configurations. MAE and maximum absolute error are reported per state variable. They give a more readable picture of the typical and worst-case deviations. Training time captures how much it costs to fit a given architecture. Inference time captures how much it costs to predict the full trajectory after training. Together, these five metrics support the accuracy vs cost trade-off analysis that our research question requires.

Table 3.4: Evaluation Metrics Used for Each Trained Model

Metric	**Purpose**
MSE (per variable and combined)	Primary loss. Consistent comparison across all 69 runs
MAE (per variable)	Interpretable error magnitude in original physical units
Maximum absolute error	Worst-case deviation along the trajectory
Training time (seconds)	Computational cost of fitting a given architecture
Inference time (seconds)	Cost of predicting the full 1001-point trajectory after training

3.2.7 Alternative Solutions Considered

We looked at several alternatives before settling on the plain feedforward ANN with a pure MSE loss. Here is what we rejected and why.

A physics-informed neural network (PINN) would add a residual term to the loss and bring back the hyperparameter balancing problem described in Chapter 2. It would also blur the line between our work and the existing PinnDE study. So we dropped it.

A DeepONet operator-learning approach was rejected for the same reason. PinnDE already evaluates a DeepONet on the same equation. Reproducing that design would not fill the architecture-selection gap we identified.

We considered a Neural ODE formulation because it represents a different paradigm. But Neural ODEs parameterize the derivative, not the solution function. That does not answer our research question.

Finally, we thought about combining architecture search and optimizer selection into a single large grid. We rejected this because it would inflate the total number of experiments and confound the two effects we want to measure separately. The two-phase design we adopted keeps the number of runs at 69 and still produces a clean attribution of the observed differences.

3.3 Project Plan

The project plan translates the research design into a 36-week execution schedule across FYDP-I, FYDP-II, and FYDP-III. The schedule groups the work into research and writing, baseline and data preparation, implementation, analysis and validation, presentation, and submission milestones, so the timeline follows the expected progression from problem framing and numerical baseline validation to ANN experiments, result verification, report completion, and defense preparation. Figure 3.3 shows the task placement across the three 12-week trimesters, and Table 3.5 defines the identifiers used in the chart.

Figure 3.3: 36-week FYDP Gantt chart using task IDs.

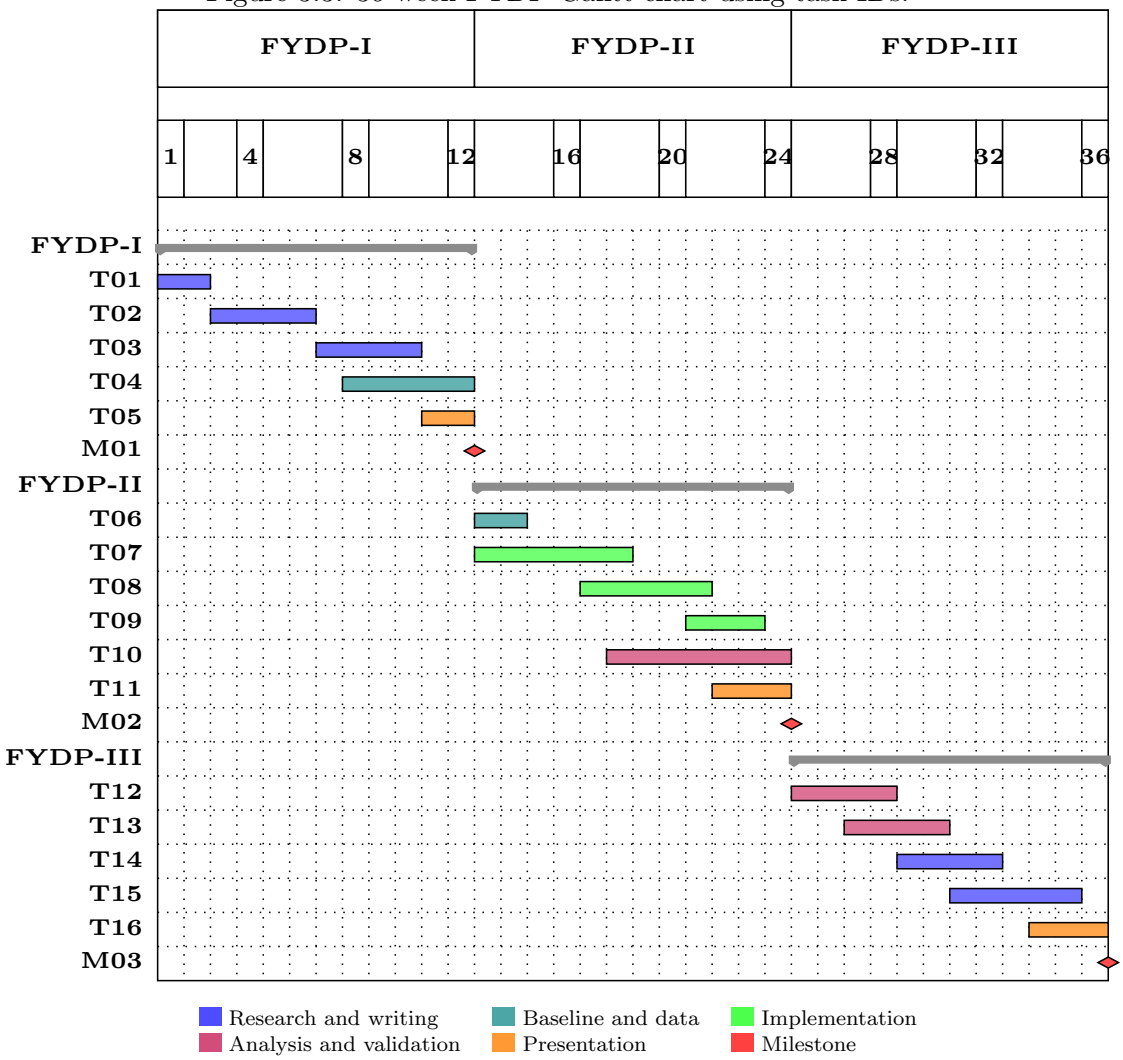

Table 3.5: Task and Milestone IDs Used in the Gantt Chart

ID	Task or Milestone	Weeks	ID	Task or Milestone	Weeks
T01	Problem framing and objectives	1–2	T10	Metrics, logging, and visualizations	18–24
T02	Literature review and gap analysis	3–6	T11	Chapter 4 draft and presentation	22–24
T03	Requirements and standards mapping	7–10	M02	FYDP-II submission milestone	24
T04	Baseline validation and dataset specification	8–12	T12	Result verification and reproducibility review	25–28
T05	Interim report and presentation	11–12	T13	Comparative analysis and final figures	27–30
M01	FYDP-I submission milestone	12	T14	Standards, constraints, and impact analysis	29–32
T06	Canonical dataset generation	13–14	T15	Final report revision and conclusion	31–35
T07	ANN training pipeline implementation	13–18	T16	Final presentation and defense preparation	34–36
T08	Phase 1 architecture search	17–21	M03	Final submission and defense	36
T09	Phase 2 optimizer comparison	21–23			

3.4 Task Allocation

The team has five members. We grouped responsibilities by skill area while also mapping each member's work across FYDP-I, FYDP-II, and FYDP-III. This avoids treating the report chapters as isolated tasks. Each member contributes to the same research pipeline from planning to final defense, but with different primary ownership.

Table 3.6: Task Allocation Across Team Members and FYDP Segments

Member	FYDP-I Responsibilities	FYDP-II Responsibilities	FYDP-III Responsibilities
Ikramul Hasan Moral	Lead numerical baseline validation; verify the Lorenz-1960 equation setup; define data-generation requirements.	Implement preprocessing, model definitions, training loop, and experiment runner for the 69 planned runs.	Recheck reproducibility, archive scripts and results, and support technical defense of the implementation.
Md. Abu Bakar	Support requirements analysis and metric selection; review literature related to evaluation criteria.	Build the evaluation pipeline; compute MSE, MAE, maximum error, training time, and inference time.	Consolidate final tables; verify result consistency against the requirements; support analysis discussion.
Samiur Rahman Omlan	Prepare methodology diagrams and initial experimental workflow documentation.	Produce convergence plots, solution trajectory plots, and architecture-comparison figures.	Finalize visual evidence for Chapter 4 and presentation slides; support result interpretation.
Fariha Islam	Lead literature review, gap analysis, citation tracking, and Chapters 1–2 drafting.	Maintain citation consistency while implementation details are added to Chapter 4.	Revise the final report for academic tone, citation coverage, and defense-readiness.
Md. Touhidul Islam	Compile Chapter 3 planning material, task allocation, and interim-report structure.	Coordinate weekly documentation, report updates, and experiment-log summaries.	Compile the final report, Chapter 5 constraints, Chapter 6 conclusion, and final presentation narrative.

All members participate in supervisor meetings, progress journals, internal result reviews, and FYDP presentations. Shared review is necessary because an error in the solver, dataset, model configuration, or interpretation would affect the whole research conclusion.

3.5 Summary

This chapter turned the research question from Chapter 1 and the literature gap from Chapter 2 into a concrete experimental design. The functional and nonfunctional requirements define what the pipeline must produce and the quality it must hit. The context diagram and Level 1 data flow diagram place the pipeline in its research setting.

The methodology is a two-phase comparative study. Phase 1 runs a 60-configuration architecture search across depth, width, and activation function. Phase 2 runs a 9-configuration optimizer comparison on the three best architectures. That gives us 69 controlled experiments in total. We generate and verify reference data across two independent solvers. We hold training, loss, and evaluation settings constant across all runs so that observed differences trace back to the variables under study.

The project plan maps the work onto the three FYDP segments. The task allocation assigns ownership of implementation, analysis, and writing to specific team members. The next chapter reports the implementation of this design and the experimental results from the 69 runs.

Chapter 4

Implementation and Results

Chapter 5

Standards and Design Constraints

5.1 Compliance with the Standards

5.2 Design Constraints

5.3 Cost Analysis

5.4 Complex Engineering Problem

This project is a complex engineering problem. It is not just writing code. We need to understand math, build numerical solvers, design neural networks, and compare results carefully. The Lorenz-1960 system is nonlinear and coupled, so solving it with an ANN requires real engineering decisions, not just running a script.

5.4.1 Complex Problem Solving

Table 5.1 shows how this project maps to the complex problem solving categories.

Table 5.1: Mapping with complex problem solving.

P1 Dept of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
✓	✓	✓	×	×	×	✓

P1: Depth of Knowledge — Applicable (✓)

We need to know many things at once: how ODEs work, how numerical solvers like Runge-Kutta produce reference solutions, how neural networks learn, how optimizers like Adam update weights, and how to measure errors. Knowing just one of these is not enough. They all connect together.

P2: Range of Conflicting Requirements — Applicable (✓)

We face trade-offs. A bigger network may give better accuracy but takes longer to train and may overfit. A faster training setup may be unstable. A model that works great once may not work again with a different random seed. We cannot get the best of everything at the same time, so we have to find a good balance.

P3: Depth of Analysis — Applicable (✓)

We do not just run the code and check if it works. We compare many ANN setups, look at error numbers, study how training behaves, check if results are stable, and explain why one setup works better than another. That takes real analysis.

P4: Familiarity of Issues — Not Applicable (×)

The Lorenz-1960 system is a well-known benchmark, and the techniques we use Runge-Kutta solvers, feedforward ANNs, Adam optimizer are all established methods. Although our specific combination is novel, the underlying issues are not unfamiliar to practitioners. So this criterion does not apply.

P5: Extent of Applicable Codes — Not Applicable (×)

Our project is a computational research study, not a commercial or safety-critical system. There are no industry standards, regulations, or professional codes that constrain how we design the ANN or evaluate its accuracy. So this criterion does not apply.

P6: Extent of Stakeholder Involvement — Not Applicable (×)

The only stakeholders are the student researchers and the academic supervisors.There are no external clients, end-users, or regulatory bodies whose conflicting needs must be balanced.All decisions are based on scientific reasoning, not stakeholder demands.So this criterion does not apply.

P7: Interdependence — Applicable (✓)

Every part of this project depends on the other parts. The solver makes the training data. The data affects how the ANN learns. The ANN design affects the results. The optimizer affects training. If any one part goes wrong, the whole result is affected.

5.4.2 Engineering Activities

Table 5.2 shows how this project maps to the complex engineering activity categories.

Table 5.2: Mapping with complex engineering activities.

A1 Range of resources	A2 Level of Interaction	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
✓	✓	✓	×	✓

A1: Range of Resources — Applicable (✓)

We use many different tools and knowledge sources: ODE math, Runge-Kutta solvers, SciPy, PyTorch, NumPy, Matplotlib, and 13 research papers. No single tool or source is enough to complete this project.

A2: Level of Interaction — Applicable (✓)

The parts of this project talk to each other. The solver gives data to the ANN. The ANN design affects training. The training setup affects results. The evaluation depends on everything before it. You cannot work on one part without thinking about the others.

A3: Innovation — Applicable (✓)

No one has done a systematic comparison of plain ANN architectures on the Lorenz-1960 system before. Most existing work uses PINNs or hybrid methods [4]. We test a different idea: can a plain ANN, without physics in the loss function, still solve this problem well? That is the new part.

A4: Consequences for Society and Environment — Not Applicable (×)

Our project is a purely academic study that trains small neural networks on a personal computer. It does not produce any physical product, does not affect public health or safety, and uses minimal computational resources. The Lorenz-1960 system is a theoretical benchmark, so our results do not drive any real-world decisions. This criterion does not apply.

A5: Familiarity — Applicable (✓)

This is not a standard textbook problem. The Lorenz-1960 system is nonlinear and chaotic. There is no ready-made ANN setup that we can just use. We have to figure out the best

architecture through experiments, and the results need careful interpretation because low training loss does not always mean the solution is correct everywhere.

5.4.3 Knowledge and Attitude Profile

Table 5.3 shows how this project maps to the knowledge and attitude profile categories.

Table 5.3: Mapping with knowledge and attitude profile.

WK1 Natural Sciences	WK2 Math & Analysis	WK3 Eng. Fundamentals	WK4 Specialist Knowledge	WK5 Resource Use	WK6 Eng. Practice	WK7 Role in Society	WK8 Research Lit.	WK9 Ethics
✓	✓	✓	✓	×	✓	×	✓	✓

WK1: Natural Sciences — Applicable (✓)

Our project requires a solid understanding of the natural sciences, specifically the physics behind the Lorenz-1960 system, which is a simplified meteorological model representing atmospheric convection. We must understand the theory governing nonlinear coupled ODEs to model these physical phenomena correctly.

WK2: Mathematics and Numerical Analysis — Applicable (✓)

This project heavily relies on conceptually based mathematics and numerical analysis. We use advanced calculus for ODEs, the Runge-Kutta method for generating reference numerical solutions, and statistics for evaluating the mean squared error of our ANN predictions. Data analysis is central to evaluating training convergence and model accuracy.

WK3: Engineering Fundamentals — Applicable (✓)

We apply a systematic formulation of engineering fundamentals by taking a well-defined mathematical problem and systematically solving it through computational means. The process of translating the Lorenz-1960 mathematical equations into a computational model requires core engineering methodology.

WK4: Specialist Knowledge — Applicable (✓)

Our work operates at the forefront of the discipline by exploring the capabilities of artificial neural networks in solving complex ODE systems. This requires specialist engineering knowledge in deep learning frameworks like PyTorch, optimization algorithms like Adam, and network architectures.

WK5: Engineering Design (Sustainability) — Not Applicable (×)

This project is a computational research study on a theoretical mathematical benchmark. It does not involve physical resource use, environmental impact assessments, or whole-life cost considerations. Therefore, this criterion does not apply.

WK6: Engineering Practice (Technology) — Applicable (✓)

We extensively use modern engineering practice and technology. We utilize programming languages and scientific computing libraries (SciPy, NumPy, Matplotlib) that are standard practice in the field of computational engineering and data science.

WK7: Role of Engineering in Society — Not Applicable (×)

The Lorenz-1960 system benchmark does not directly impact public safety, societal issues, or sustainable development. It is an academic exercise in algorithm evaluation, so there are no direct societal stakeholders whose safety must be considered. This criterion does not apply.

WK8: Engagement with Research Literature — Applicable (✓)

We conducted an extensive literature review, analyzing 13 research papers, to understand the current state-of-the-art in PINNs and standard ANNs for ODE solving. We applied critical thinking to identify the research gap—specifically, evaluating plain ANNs on the Lorenz-1960 system without physics-informed loss functions.

WK9: Ethics and Professional Conduct — Applicable (✓)

We adhere to professional ethics and academic integrity by properly citing all research papers and methodologies used. We ensure our experiments are reproducible, and our results are reported honestly without manipulation.

5.5 Summary

Chapter 6

Conclusion

This project investigates whether plain feedforward artificial neural networks can approximate the solution of the Lorenz-1960 ordinary differential equation system when trained only on high-accuracy numerical reference data. The motivation comes from a practical and academic tension. Classical numerical solvers such as Runge-Kutta methods provide dependable reference solutions, but they solve the system step by step on a fixed grid. Neural approaches can instead learn a continuous input-output mapping from time to the state variables, but recent work on differential equations often relies on physics-informed losses or operator-learning architectures [4, 5, 6]. This study therefore keeps a narrow scope: a data-driven feedforward ANN is trained to approximate $(x(t), y(t), z(t))$ for the Lorenz-1960 initial value problem, without adding ODE residual terms to the loss function.

The research design centers on a validated numerical baseline and a controlled architecture comparison. The Lorenz-1960 formulation, parameters, initial conditions, and time interval follow the system described in Chapter 2 from the PinnDE reference problem [4]. The reference trajectory is generated and checked through two numerical solvers, a custom RK4 implementation and SciPy DOP853, before it is used as training and validation data. The ANN study is then divided into two phases. Phase 1 compares depth, width, and activation function while holding the optimizer fixed. Phase 2 compares Adam, L-BFGS, and SGD with momentum on the best Phase 1 architectures. This separation is intended to make the architecture and optimizer effects easier to interpret.

The expected contribution of the work is an empirical benchmark for plain ANN approximation of the Lorenz-1960 system. Existing studies show that neural networks can solve or approximate differential equations under several formulations [8, 11, 12], and physics-informed approaches have been applied to Lorenz-type problems [4, 18]. The specific academic value of this project is its focus on a simpler question: how far a standard feedforward ANN can go when architecture choices are varied systematically and the training target is a verified numerical solution. The answer can help clarify whether better plain-ANN design is sufficient for this benchmark or whether physics-informed constraints remain necessary for higher accuracy and reliability.

At the current report stage, the supported contribution is the validated baseline de-

sign, the dataset and training protocol, and the planned evaluation framework. The report does not claim a final best ANN architecture unless the corresponding experiment records, metric tables, and plots are present in Chapter 4. This limitation is deliberate. A conclusion based on unsupported accuracy claims would weaken the study. Final claims about the best depth, width, activation function, or optimizer should be added only after the complete experimental evidence has been generated and reviewed.

After the planned comparison is completed, future work should extend the study in three directions. First, the best configuration should be tested under repeated random seeds to measure stability. Second, the trained model should be evaluated on denser time grids or additional initial conditions to assess generalization beyond the current trajectory. Third, the plain ANN results should be compared more directly with physics-informed and operator-learning baselines from the literature. These extensions would help determine whether the architecture selected in this project is only effective for one controlled trajectory or whether it can support broader neural approximation of nonlinear ODE systems.

References

[1] Facundo Bre, Juan Gimenez, and Víctor Fachinotti. Prediction of wind pressure coefficients on building surfaces using artificial neural networks. *Energy and Buildings*, 158, 11 2017.

[2] Weida Zhai, Dongwang Tao, and Yuequan Bao. Parameter estimation and modeling of nonlinear dynamical systems based on runge–kutta physics-informed neural network. *Nonlinear Dynamics*, 111:1–14, 10 2023.

[3] Edward N. Lorenz. Maximum simplification of the dynamic equations. *Tellus*, 12(3):243–254, 1960.

[4] Jason Matthews and Alex Bihlo. Pinnde: physics-informed neural networks for solving differential equations. *arXiv preprint arXiv:2408.10011*, 2024.

[5] Liam LH Lau and Denis Werth. Oden: A framework to solve ordinary differential equations using artificial neural networks. *arXiv preprint arXiv:2005.14090*, 2020.

[6] Atmane Babni, Ismail Jamiai, and José Alberto Rodrigues. Error estimates and generalized trial constructions for solving odes using physics-informed neural networks. *Mathematical and Computational Applications*, 30(6):127, 2025.

[7] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[8] Sourabh Kumar Dubey, Raghvendra Singh, and Hibah Islahi. Solving ordinary differential equations using artificial neural networks: A comprehensive approach. *Journal of Computational Analysis & Applications*, 33(6), 2024.

[9] Khadeejah James Audu, Olalekan Abdrasheed Kayode, Fajuyi Samuel, Mubarak Inuolaji, and Yahaya Yusuph Amuda. Neural network solutions for first-order differential equations: A physics-informed perspective. *Journal of Engineering and Basic Sciences*, 5:1–13, 2026.

[10] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[11] Susmita Mall and Snehashish Chakraverty. Comparison of artificial neural network architecture in solving ordinary differential equations. *Advances in Artificial Neural Systems*, 2013(1):181895, 2013.

[12] Roseline N Okereke, Olaniyi S Maliki, and Ben I Oruh. A novel method for solving ordinary differential equations with artificial neural networks. *Applied Mathematics*, 12(10):900–918, 2021.

[13] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[14] Khadeejah James Audu, Marshal Benjamin, Umar Mohammed, and Yusuph Amuda Yahaya. Utilizing the artificial neural network approach for the resolution of first-order ordinary differential equations. *Malaysian Journal of Science and Advanced Technology*, pages 210–216, 2024.

[15] V Murugesh, M Priyadharshini, Yogesh Kumar Sharma, Umesh Kumar Lilhore, Roobaea Alroobaea, Hamed Alsufyani, Abdullah M Baqasah, and Sarita Simaiya. A novel hybrid framework for efficient higher order ode solvers using neural networks and block methods. *Scientific Reports*, 15(1):8456, 2025.

[16] Danang A Pratama, Maharani A Bakar, NB Ismail, and M Mashuri. Ann-based methods for solving partial differential equations: a survey. *Arab Journal of Basic and Applied Sciences*, 29(1):233–248, 2022.

[17] Xiao Xiong. New designed loss functions to solve ordinary differential equations with artificial neural network. *arXiv preprint arXiv:2301.00636*, 2022.

[18] Muhammad Naeem Aslam, Muhammad Waheed Aslam, Muhammad Sarmad Arshad, Zeeshan Afzal, Murad Khan Hassani, Ahmed M Zidan, and Ali Akgül. Neurocomputing solution for lorenz differential equations through artificial neural networks integrated with pso-nna hybrid meta-heuristic algorithms: a comparative study. *Scientific Reports*, 14(1):7518, 2024.

[19] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.